

Computergrafik

PD Prof. Dr. rer nat.
Marco Block-Berlitz

Sommersemester 2026



Entwicklung eines Wassershaders Konvolution und Blur-Effekt

„Das Wasser nimmt jede Form an, in die man es gießt, und doch bleibt es sich selbst treu.“ (sinngemäß nach Katsushika Hokusai)

PD Prof. Dr. Marco Block-Berlitz

Interaktives Wasser – Projektetappen

Wasser hat in diesem Kontext schon immer eine besondere Anziehungskraft ausgeübt, symbolisiert es doch den Ursprung und das Grundbedürfnis des Lebens. Das folgende Projekt widmet sich dem Motto „Es war einmal das Wasser ...“ und wird Schritt für Schritt in die **Shaderprogrammierung mit GLSL** einführen und und so ganz nebenbei – die notwendigen mathematischen und physikalischen Konzepte behandeln. Beiläufig sehen wir instruktive Shaderbeispiele und machen uns mit GLSL vertraut.

Blur-Effekt

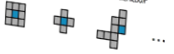
Blur-Effekt: Glättung durch Konvolution

Die **Konvolution** ist ein sehr häufig eingesetztes Verfahren in der Bildverarbeitung mit sehr unterschiedlichen Anwendungsgebieten. Dabei wird aus einem gegebenen Bild ein neues erstellt und es werden für jedes Pixel Informationen über seine Nachbarn herangezogen, um die neue Pixelfarbe zu bestimmen.

Ursprungsbild t Zielbild $t+1$



Ganz unterschiedliche Kernel sind denkbar



Der **Initialwert** beschreibt sich aus der Summe der Nachbarn mit dem Ursprungspixel und der anschließenden Division durch die Anzahl der summierten Pixel:

$$m_{t+1} = (a + b + c + d + m_t) \cdot 0.2$$

32

31

Ziele und Inhalt

- Shaderprogrammierung mit Java, OpenGL, GLSL und WebGL
 - Motivation zum Wasserprojekt
 - Ausblick auf die Entwicklung
 - Projektetappen: Interaktive Wassersimulation
 - Projektziel – Stufe 1
 - Blur-Effekt: Glättung durch Konvolution
 - Blur-Effekt: Konvergenz bei Kontinuität
 - Lokale Operationen in der Bildverarbeitung
 - Dämpfung durch Maus-Interaktion
 - Pseudocode gedämpfter Mittelwertfilter
 - Implementierung in Java
-
- Fragestellungen zum Vorlesungsteil
 - Praktische Aufgaben





Verschiedene Wege führen zur
Shaderprogrammierung

Unser Shader-Setup für diese Veranstaltung

Eine wichtige Triebfeder der letzten Jahre ist an dieser Stelle sicherlich die Spieleindustrie, mit ihrem massiven Einsatz von synthetischen Welten und dem Bedürfnis, die Welten bestimmter Genres immer realitätsnäher zu gestalten. Dabei liegt der Fokus besonders auf echtzeitbasierten Renderingprozessen.



Unser Shader-Setup für diese Veranstaltung

Eine wichtige Triebfeder der letzten Jahre ist an dieser Stelle sicherlich die Spieleindustrie, mit ihrem massiven Einsatz von synthetischen Welten und dem Bedürfnis, die Welten bestimmter Genres immer realitätsnäher zu gestalten. Dabei liegt der Fokus besonders auf echtzeitbasierten Renderingprozessen.





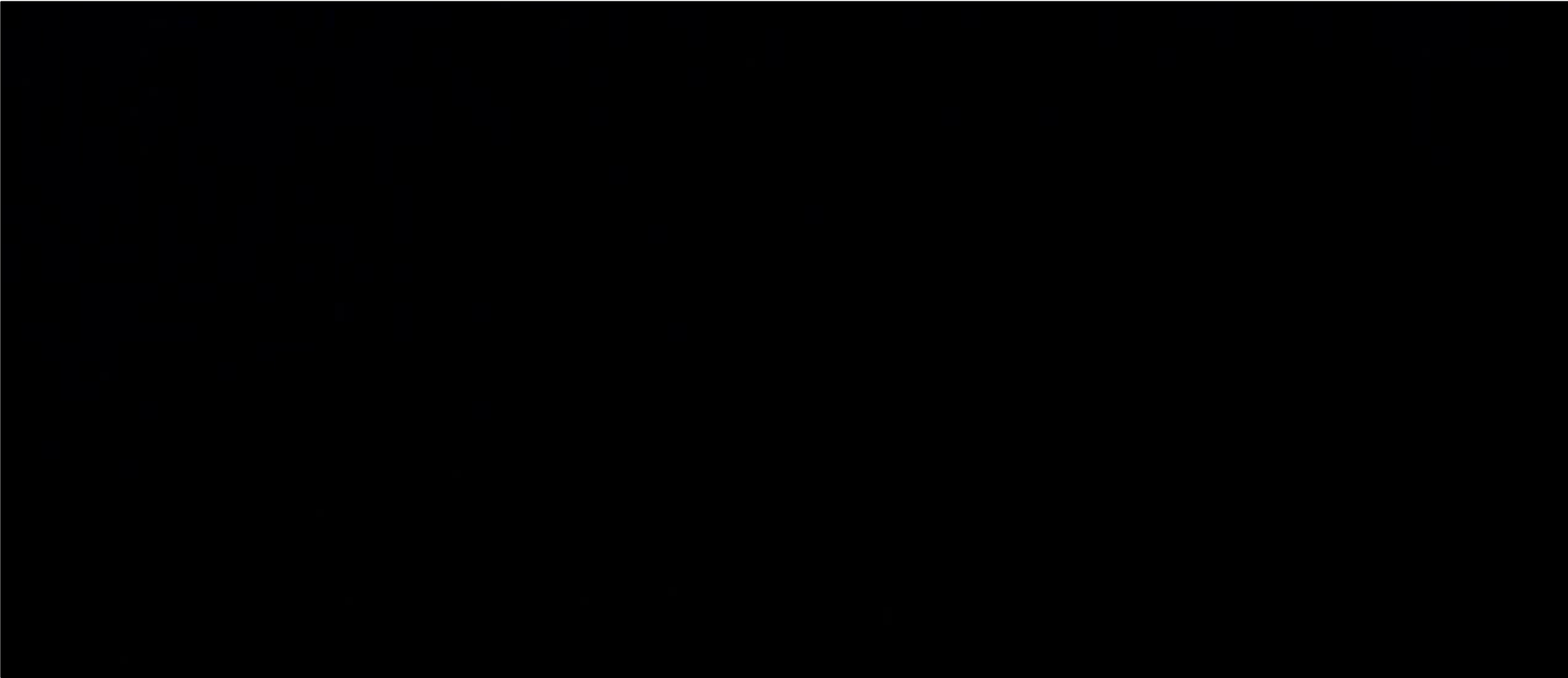
Projekt „Es war einmal das Wasser ...“

Motivation Dreamaquarium



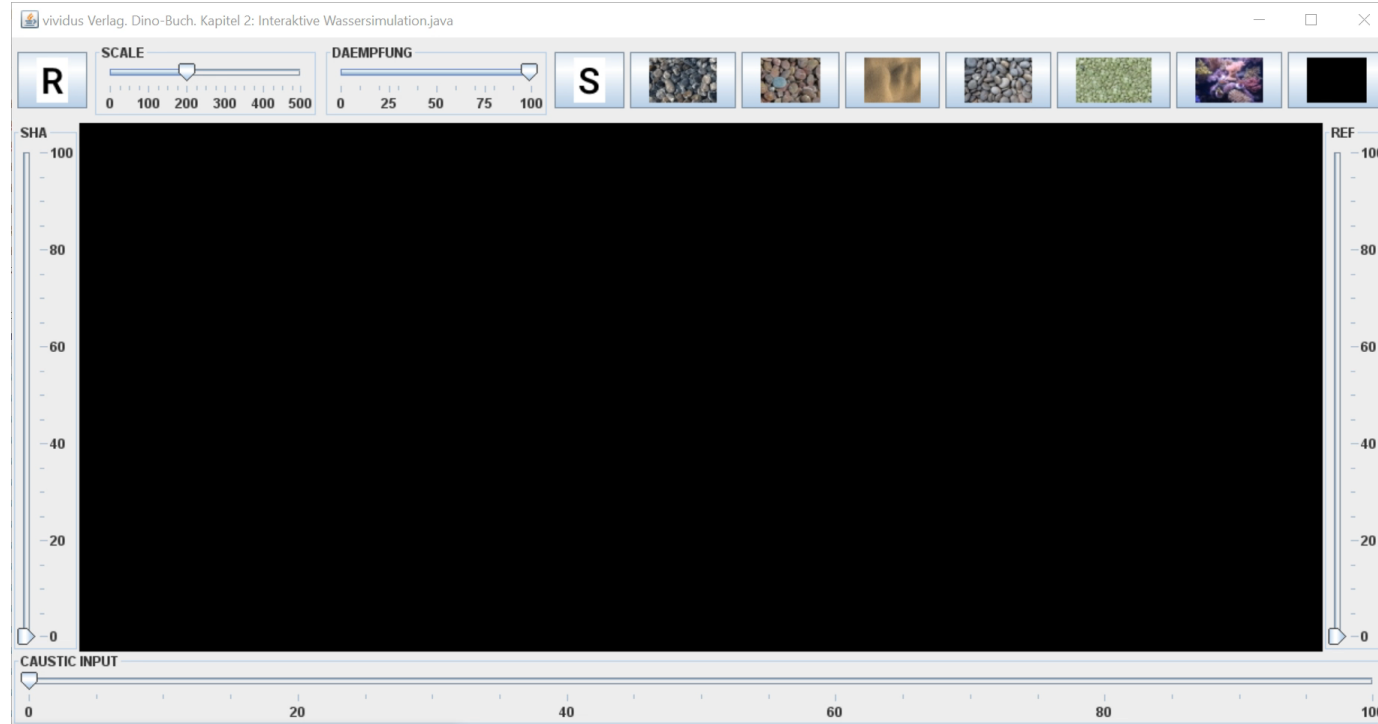
[1] Alan Kaplers, Dream Aquarium (<https://www.dreamaquarium.com>): Dream Aquarium - 2 Hours - 8 Tanks (4K)
<https://www.youtube.com/watch?v=HHi8qOtHnhE>

Microsoft Surface – Lagoon



Kleiner Ausblick auf unsere Entwicklung

Schauen wir uns das Ergebnis der experimentellen Wassersimulation an:



Video

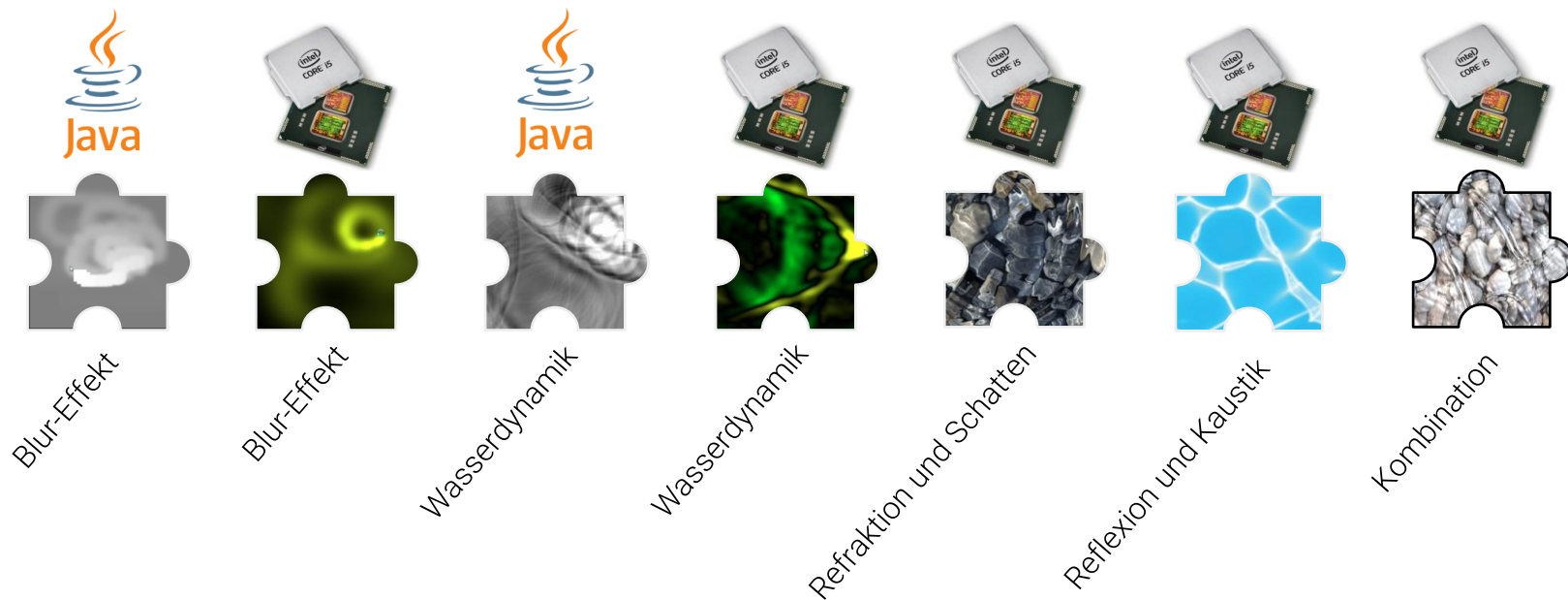


Programmcode

kapitel02/Interaktive-
Wassersimulation.java

Interaktives Wasser – Projektetappen

Wasser hat in diesem Kontext schon immer eine besondere Anziehungskraft ausgeübt, symbolisiert es doch den Ursprung und das Grundbedürfnis des Lebens. Das folgende Projekt widmet sich dem Motto „Es war einmal das Wasser ...“ und wird Schritt für Schritt in die [Shaderprogrammierung mit GLSL](#) einführen und und so – ganz nebenbei – die notwendigen mathematischen und physikalischen Konzepte behandeln. Beiläufig sehen wir instruktive Shaderbeispiele und machen uns mit GLSL vertraut.





Konvolution und Blur-Effekt

Projektziel – Stufe 1

Bei der ersten Projektstufe ist vorgesehen, dass ein weißes Viereck jedes Mal zu sehen ist und damit bei jedem Zeichenvorgang mehr „Energie“ in das Bild getragen wird.

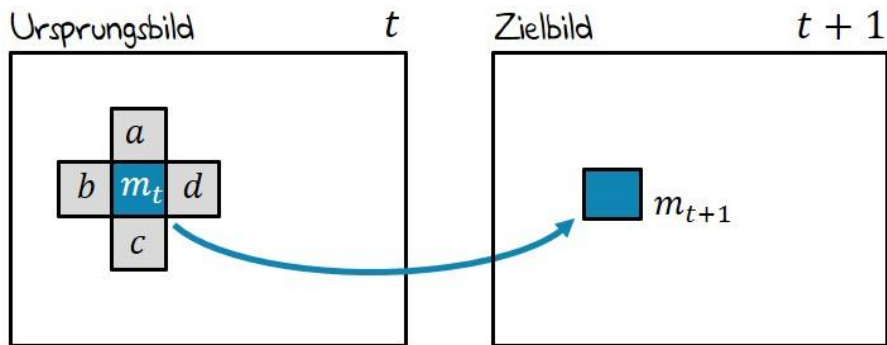


Projektstufe 1

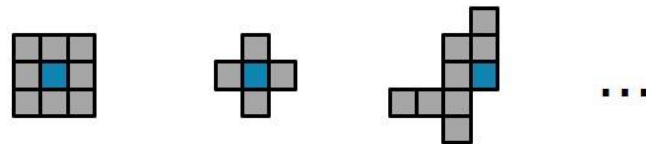
- wir haben einen einfarbigen, grauen Hintergrundbereich
- mit einem weißen Vordergrundobjekt (Viereck)
- das Vordergrundobjekt soll mit der Maus bewegt werden können
- die Zwischenpositionen sollen kontinuierlich verblassen und in den Hintergrund übergehen

Blur-Effekt: Glättung durch Konvolution

Die **Konvolution** ist ein sehr häufig eingesetztes Verfahren in der Bildverarbeitung mit sehr unterschiedlichen Anwendungsgebieten. Dabei wird aus einem gegebenen Bild ein neues erstellt und es werden für jedes Pixel Informationen über seine Nachbarn herangezogen, um die neue Pixelfarbe zu bestimmen.



Ganz unterschiedliche Kernel sind denkbar



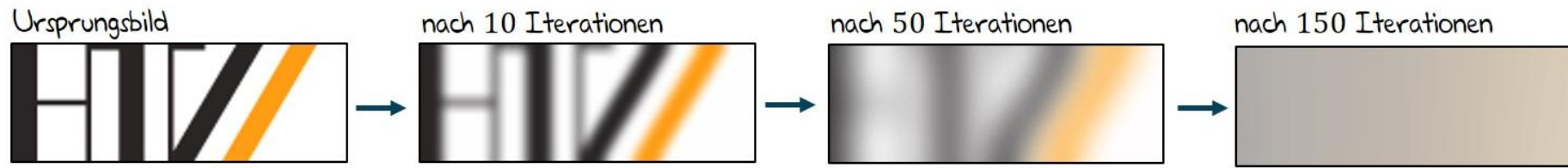
Der **Mittelwert** berechnet sich aus der Summe der Nachbarn mit dem Ursprungspixel und der anschließenden Division durch die Anzahl der summierten Pixel:

$$m_{t+1} = (a + b + c + d + m_t) \cdot 0.2$$

Blur-Effekt: Konvergenz bei Kontinuität

Dieses Verfahren wird auch als **Mittelwertfilter** bezeichnet. Derartige Methoden kommen häufig dann zum Einsatz, wenn Rauscheffekte im Bild vorhanden sind oder Informationen aus einem Bild abstrahiert werden sollen.

Sollten wir das gleiche Verfahren immer wieder auf das resultierende Bild anwenden, wird es nach vielen Iterationen immer unschärfer und schließlich zu einem einfarbigen Bild konvergieren, das die **durchschnittliche Farbe** des ursprünglichen Bildes aufzeigt:



Lokale Operationen in der Bildverarbeitung

Verschiedene **Filter** (linear oder nicht-linear) können auf ein Bild I durch **Faltung** angewendet werden [1].

Mittelwert-Filter explizit (3×3 -Kernel)

$$I'(u, v) = \frac{1}{9} \sum_{j=-1}^1 \sum_{i=-1}^1 I(u + i, v + j)$$

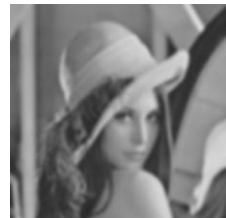


Mittelwert-Filter + Filtermaske (3×3 -Kernel)

$$I'(u, v) = \sum_{j=-1}^1 \sum_{i=-1}^1 I(u + i, v + j) \cdot F(i, j)$$

Tiefpass-Filter (3×3 -Kernel)

$$F_{\text{tiefpass}} = \begin{bmatrix} 0.011 & 0.084 & 0.011 \\ 0.084 & 0.619 & 0.084 \\ 0.011 & 0.084 & 0.011 \end{bmatrix}$$



Hochpass-Filter (3×3 -Kernel)

$$F_{\text{hochpass}} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$



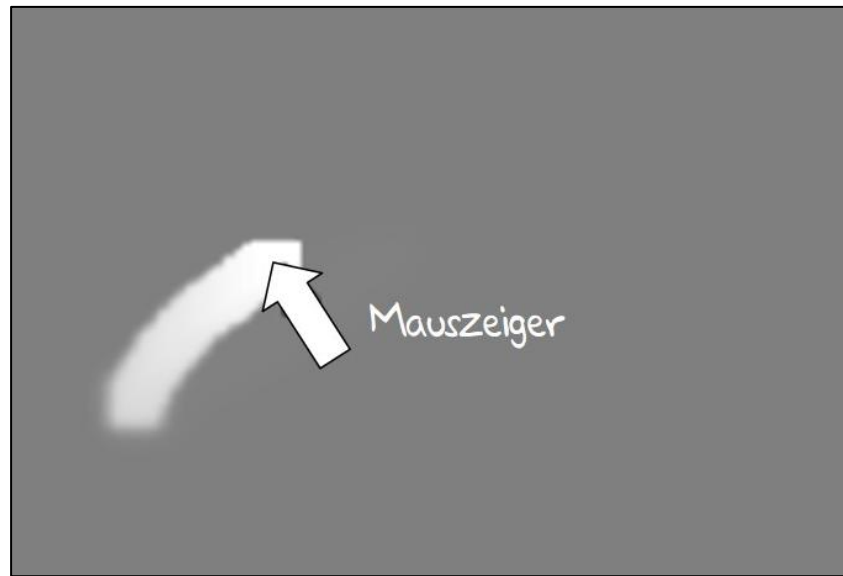
Dämpfung durch Maus-Interaktion

Da wir mit dem System möglicherweise über eine längere Zeit interagieren wollen, müssen wir die Konvolutions-Formel aus dem vorhergehenden Abschnitt etwas erweitern. Wenn wir das nicht tun, wird sich die Helligkeit des Hintergrundes zu jedem Zeitpunkt leicht erhöhen und schließlich **zu der Farbe Weiß konvergieren**, da das Vordergrundobjekt jedes Mal neu gezeichnet wird.

Zu dem gewünschten Grauwert G , der die Hintergrundfarbe darstellt, wird eine **gedämpfte Version des Mittelwertfilters** hinzuaddiert:

$$m_{t+1} = G + (a + b + c + d + m_t) \cdot 0.2 \cdot D$$

Der **Dämpfungsfaktor** D ist dabei ein Wert nahe bei (aber immer kleiner als) 1.

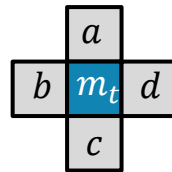


Pseudocode für gedämpften Mittelwertfilter

Angenommen, es wird im folgenden Beispiel kontinuierlich im Bild **source** an der Stelle der Mausposition ein weißes Viereck gezeichnet, dann können wir den Blur-Effekt für das Bild **target** im **Pseudocode** wie folgt formulieren:

```
for all target pixel (x, y) {  
    // Kernel für 4-er Nachbarschaft  
    target[x, y] = source[x-1, y] +  
                    source[x+1, y] +  
                    source[x, y] +  
                    source[x, y-1] +  
                    source[x, y+1];  
  
    // Mittelwert  
    target[x, y] = target[x, y] * 0.2f;  
  
    // Gewünschter Hintergrundgrauwert und Dämpfung  
    target[x, y] = G + target[x, y] * D;  
}
```

$$m_{t+1} = a + b + c + d + m_t$$



$$m_{t+1} = (a + b + c + d + m_t) \cdot 0.2$$

$$m_{t+1} = G + (a + b + c + d + m_t) \cdot 0.2 \cdot D$$

Implementierung in Java

Für die Vorbereitung wollen wir ein Frame erzeugen und bei jedem Mauszug weiß gefüllte Vierecke an den Mauspositionen zeichnen. Für den Blur-Effekt erzeugen wir zwei Speicherbereiche **surfaceA** und **surfaceB**.

```
private void initialize() {
    surfaceA = new float[WIDTH * HEIGHT];
    surfaceB = new float[WIDTH * HEIGHT];
    pixels = new int[WIDTH * HEIGHT];
    mis = new MemoryImageSource(WIDTH, HEIGHT, pixels, 0, WIDTH);
    mis.setAnimated(true);

    f = new JFrame();
    final JComponent viewer = new JComponent() {
        private static final long serialVersionUID = 1L;
        Image im = createImage(mis);
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            g.drawImage(im, 0, 0, getWidth(), getHeight(), 0, 0,
                BlurEffektCPU.WIDTH, BlurEffektCPU.HEIGHT, this);
        }
    };
    f.getContentPane().add(viewer);
    f.setBounds(100, 100, 2 * WIDTH, 2 * HEIGHT);
    f.setVisible(true);
    f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}
```

```
mouseCoords = new Point();
viewer.addMouseMotionListener(new MouseAdapter() {
    public void mouseDragged(MouseEvent e) {
        mouseCoords.setLocation(e.getX() * WIDTH / viewer.getWidth(),
            e.getY() * HEIGHT / viewer.getHeight());
    }
});
```

Implementierung in Java

Für die Vorbereitung wollen wir ein Frame erzeugen und bei jedem Mauszug weiß gefüllte Vierecke an den Mauspositionen zeichnen. Für den Blur-Effekt erzeugen wir zwei Speicherbereiche **surfaceA** und **surfaceB**.

```
private void initialize() {
    surfaceA = new float[WIDTH * HEIGHT];
    surfaceB = new float[WIDTH * HEIGHT];
    pixels = new int[WIDTH * HEIGHT];
    mis = new MemoryImageSource(WIDTH, HEIGHT, pixels, 0, WIDTH);
    mis.setAnimated(true);

    f = new JFrame();
    final JComponent viewer = new JComponent() {
        private static final long serialVersionUID = 1L;
        Image im = createImage(mis);
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            g.drawImage(im, 0, 0, getWidth(), getHeight(), 0, 0,
                BlurEffektCPU.WIDTH, BlurEffektCPU.HEIGHT, this);
        }
    };
    f.getContentPane().add(viewer);
    f.setBounds(100, 100, 2 * WIDTH, 2 * HEIGHT);
    f.setVisible(true);
    f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}
```

```
mouseCoords = new Point();
viewer.addMouseMotionListener(new MouseAdapter() {
    public void mouseDragged(MouseEvent e) {
        mouseCoords.setLocation(e.getX() * WIDTH / viewer.getWidth(),
                                e.getY() * HEIGHT / viewer.getHeight());
    }
});
```

Implementierung in Java

Für die Darstellung der Pixelfarben als **int**-Wert werden wir einen kontinuierlichen Übergang verschiedener Grauwerte in Bezug auf die berechneten Glättungsdaten vom Typ **float** verwenden. Wir hatten weiterhin den Wunsch geäußert, einen **mittleren Grauwert** als Hintergrundfarbe zu sehen.

Die Funktion **colorForAmplitudeGray** übernimmt diese Transformation und wird im nachfolgenden Programmabschnitt gezeigt:

```
public static int colorForAmplitudeGray(float a) {  
    int gray = (int)((a<0 ? 0 : a)*128); // Werte kleiner 0 auf 0  
    gray = gray>128 ? 128 : gray;       // Werte größer 128 auf 128  
    gray += 127;                        // mittlerer Grauwert  
  
    return 0xFF000000 | (gray<<16) | (gray<<8) | gray;  
}
```



Implementierung in Java

Schauen wir uns den für die Konvolution relevanten Programmteil an, den die Funktion **renderLoop** bereitstellt:

```
public void renderLoop() throws InterruptedException {
    while (f.isVisible()) {
        float[] newB = surfaceA; float[] newA = surfaceB; surfaceA = newA; surfaceB = newB;

        for (int y=1,o=WIDTH+1; y<HEIGHT-1; y++,o+=2)                // Blur-Effekt
            for (int x=1; x<WIDTH-1; x++,o++)
                surfaceB[o] = ((surfaceA[o]+surfaceA[o-1]+surfaceA[o+1]+
                    surfaceA[o-WIDTH]+surfaceA[o+WIDTH])*0.2f)*D;

        int Q_SIZE = WIDTH/20;                                        // Quaderobjekt
        int posX = Math.min(Math.max(mouseCoords.x-Q_SIZE/2,1), WIDTH-Q_SIZE);
        int posY = Math.min(Math.max(mouseCoords.y-Q_SIZE/2,1), HEIGHT-Q_SIZE);

        // Alle Pixel des Quaderobjekts vollständig setzen
        for (int y=0,o=posX+posY*WIDTH, r=WIDTH-Q_SIZE; y<Q_SIZE;y++,o+=r)
            for (int x=0; x<Q_SIZE; x++,o++)
                surfaceB[o] = 1;

        for (int i=0,l=pixels.length; i<l; i++)
            pixels[i] = colorForAmplitudeGray(surfaceB[i]);

        mis.newPixels();
        Thread.sleep(10);
    }
}
```

$$m_{t+1} = (a + b + c + d + m_t) \cdot 0.2 \cdot D$$

Quader vorbereiten

Quader zeichnen

Pixelwerte aus **float** erzeugen

Implementierung direkt in Java



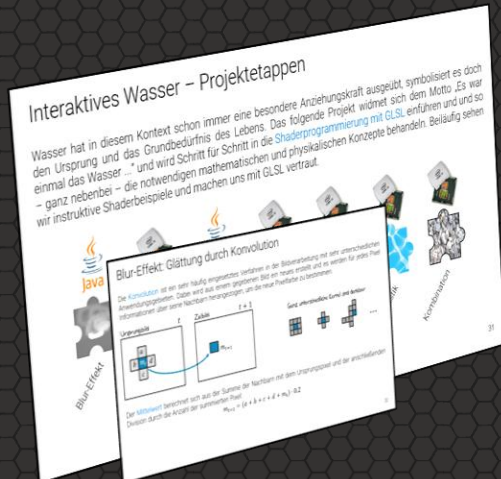
Programmcode:
kapitel02/BlurEffektCPU.java

Entwicklung eines Wassershaders

Konvolution und Blur-Effekt

Key Takeaways

- Konvolution verknüpft Pixel lokal mit ihrer Nachbarschaft
- Mittelwertfilter erzeugen Glättung und damit einen Blur-Effekt
- Wiederholte Anwendung reduziert Kontraste und verteilt Bildinformationen
- Tiefpassfilter glätten, Hochpassfilter heben Kanten hervor
- Dämpfung verhindert Aufsummierung von Energie
- Zwei Bildpuffer ermöglichen eine saubere iterative Berechnung
- Der Blur-Effekt ist Grundlage für spätere Wassersimulationen



Fragestellungen zum Vorlesungsteil

Im Anschluss an diese Veranstaltung sollten Sie die folgenden Fragen sicher beantworten können:

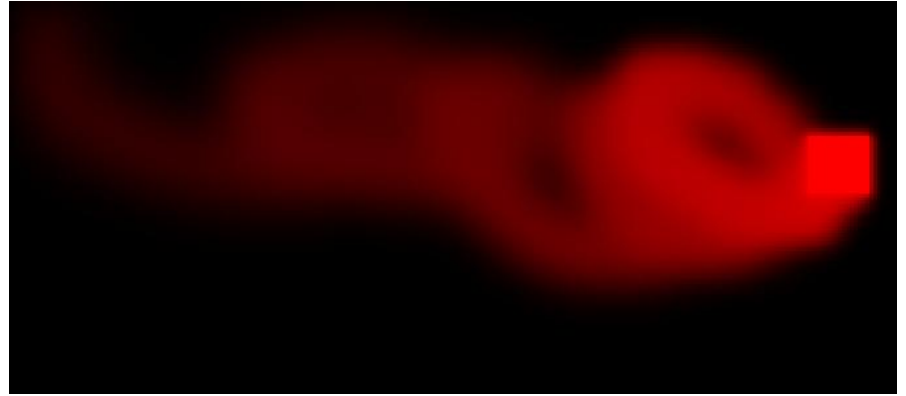
- (1) Erläutern Sie das Verfahren der Glättung durch Konvolution anhand eines Mittelwertfilters und beschreiben Sie, welche Rolle die lokale Nachbarschaft dabei spielt.
- (2) Recherche: Erläutern Sie den Unterschied zwischen linearen und nicht-linearen Filtern und geben Sie jeweils ein konkretes Beispiel aus der Bildverarbeitung. Was sind morphologische Operatoren?
- (3) Beschreiben Sie, wie sich ein Bild bei wiederholter Anwendung eines Mittelwertfilters verändert und gegen welchen Zustand es konvergiert.
- (4) Erläutern Sie den Unterschied zwischen einem klassischen Mittelwertfilter und einem gedämpften Mittelwertfilter und begründen Sie, warum die Dämpfung in einem iterativen System notwendig ist.
- (5) Warum werden in der Implementierung zwei Speicherbereiche verwendet und welches Problem würde bei der Verwendung nur eines Arrays auftreten?



Praktische Aufgaben

Folgende praktische Aufgaben sollten Sie jetzt selbstständig durchführen:

- (1) **Aufgabe 1)** [Java] Verändern Sie das Programm `BlurEffektCPU.java` so, dass nicht ein weißer Quader, sondern ein roter Quader gezeichnet wird.





Vielen Dank für die Aufmerksamkeit